

MODULE 6

ML Frameworks for Microcontrollers

Edge Impulse · emlearn · AlfES · TFLite-Micro

TinyML Course · Day 2

The problem every framework solves

TinyML challenge: **PC/host** vs **embedded/device** pipeline

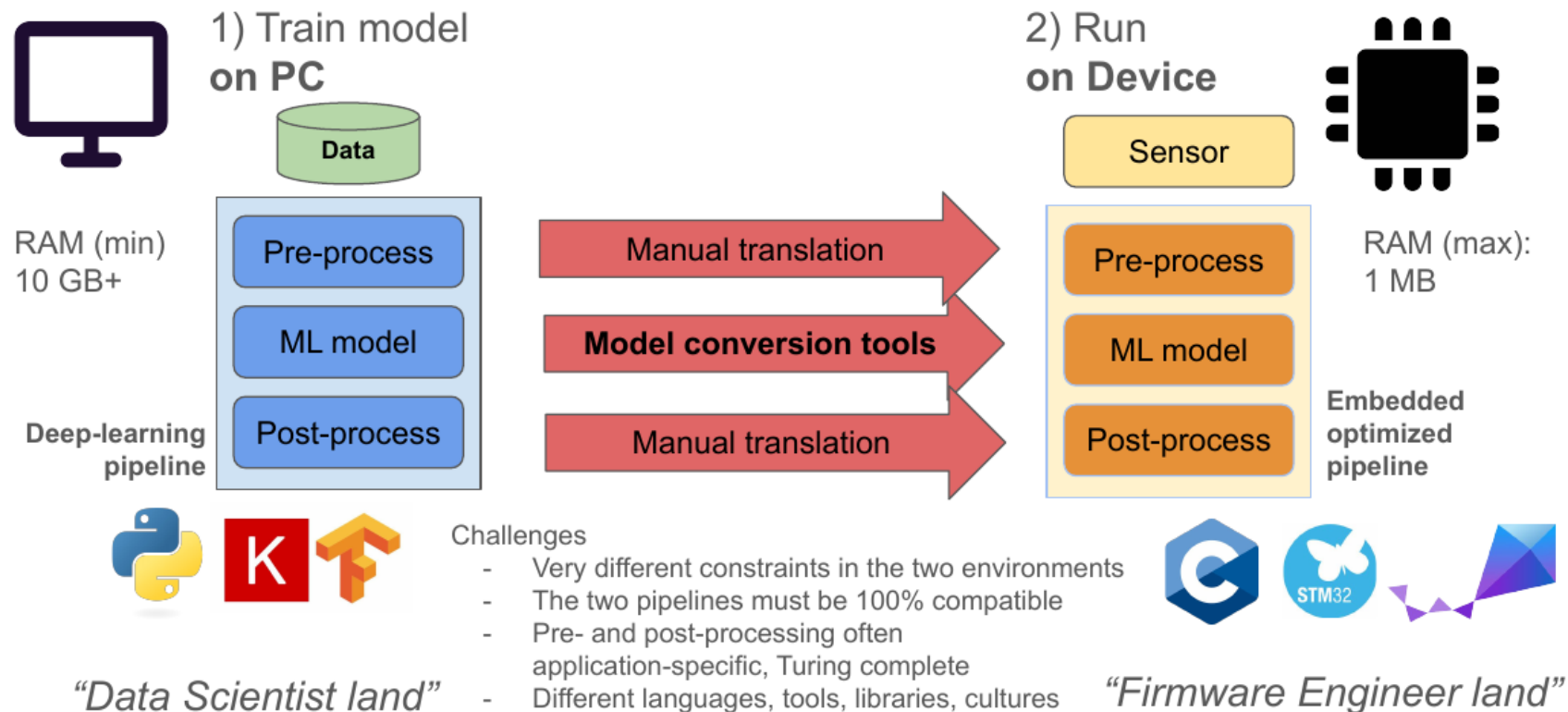


Figure: Jon Nordby, "Practical TinyML with emlearn".

Two pipelines one behaviour: train on a 10 GB+ PC in Python — run in ≤ 1 MB in C on MCU

Four (of many) frameworks

	One-liner
Edge Impulse	End-to-end SaaS: data → DSP → training → deployable C++ library
emlearn	"scikit-learn for microcontrollers": Python train → tiny portable C99 header
AlfES	Fraunhofer IMS C library: NN inference and training on-device
TFLite-Micro	Google's reference NN interpreter for MCUs; the ecosystem default

In addition: MicroPython + emlearn-micropython, CMSIS-NN (kernel layer), vendor tools (ST Cube.AI, NXP eIQ...).

Edge Impulse

- **Workflow:** Studio (web) — ingestion, labelling, DSP blocks, NN/classical training, EON-compiled C++ export; CLI tools for data forwarding
- **Models:** dense NN, 1D/2D CNN (Keras under the hood), classical blocks, K-means/GMM anomaly
- **Footprint:** EON compiler generates model as C++ (no interpreter) — RAM/flash shown *per build* in Studio
- **Licence/cost:** free developer tier; paid enterprise; **generated SDK ships with your project** — core parts Apache-2.0
- **Lock-in:** data is exportable; the *pipeline* (DSP config, training, EON) lives in their cloud



Kick this off now — EI export (~5 min, then it builds while we talk)

In **your fan project** from Module 4:

1. Impulse design → add **Spectral Analysis** + a small NN → train
2. **Deployment** → C++ library (or Arduino library) → **Build**
 - The build runs server-side — collect the .zip when we reach the lab
 - **Done when:** the download link exists (we compile it in the lab)

emlearn

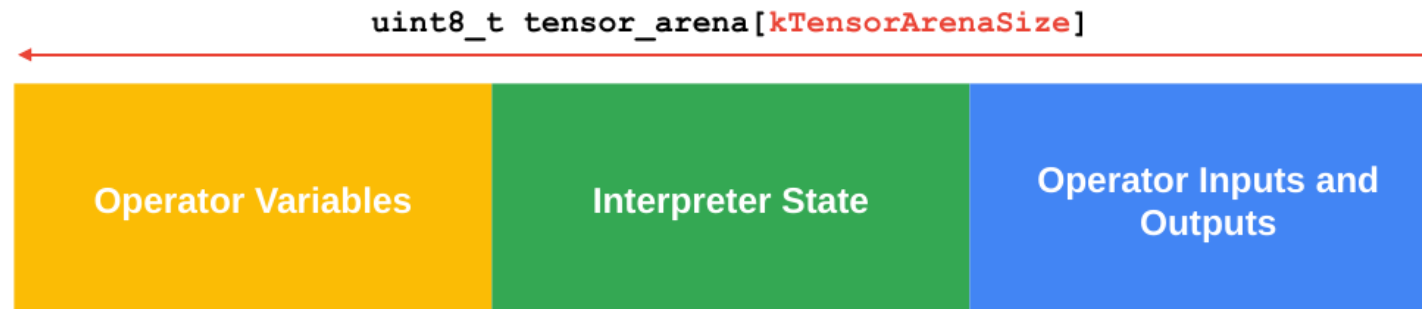
- **Workflow:** train with plain scikit-learn/Keras → `emlearn.convert(model)` → `.save()`
→ one C header → `#include`, call `predict()`
- **Models:** Decision Tree, **Random Forest**, ExtraTrees, KNN, Gaussian NB, GMM/EllipticEnvelope (anomaly), MLP
- **Footprint:** from **~2 kB flash**; no interpreter, no malloc, pure C99; integer/fixed-point support
- **Licence:** **MIT** — do anything, ship anywhere
- **Lock-in:** none worth the name — output is readable C you can version-control

AlfES

- **Workflow**: define the network **in C** (layers/weights as arrays); import Keras/PyTorch weights, or train from scratch **on the device**
- **Models**: dense feed-forward NN (f32 **and Q7 quantised**); CNN support growing
- **Footprint**: a few kB–tens of kB; benchmarks vs TFLM: **~2.1× faster, up to 3.9× less memory** (Fraunhofer workshop numbers)
- **Licence**: **AGPL-3.0** open-source, **commercial licence available from Fraunhofer**
- **Lock-in**: library API; weights are portable C arrays

TensorFlow Lite for Microcontrollers

- **Workflow:** Keras → TFLite converter (+ int8 quantisation) → FlatBuffer byte array → **interpreter** runs it on-device
- **Models:** NN only — dense, CNN, RNN subsets; ~60–100 supported operators
- **Footprint:** interpreter core ~16 kB+ flash, plus per-operator kernels + **tensor arena** (RAM, you size it)
- **Licence:** **Apache-2.0**
- **Lock-in:** low-ish — open format, but you inherit the TF toolchain and its churn



The comparison table

	Edge Impulse	emlearn	AlfES	TFLite-Micro
Licence	SaaS + SDK (parts Apache-2.0) ¹	MIT	AGPL-3.0 / commercial ¹	Apache-2.0
Train where	their cloud	your PC (sklearn)	PC or on-device	your PC (TF)
Model types	NN, CNN, classical, anomaly	trees, KNN, GMM, NB, MLP	dense NN (f32/Q7)	NN (op subset)
Feature/DSP tooling	✓ built-in blocks	✗ DIY	✗ DIY	✗ DIY
Footprint floor	~10s of kB	~2 kB	~few kB	~20 kB + arena
Runtime style	generated C++ (EON)	generated C99	C library + weight arrays	interpreter + FlatBuffer
Vendor lock-in	pipeline in cloud	none	low (API)	low (format)
Best when	speed to demo, team workflow	classical ML, tiny targets, full control	on-device learning, Structured framework	existing TF models, wide ops support

Selection criteria 1 — licence & governance

- **Can you ship it?** MIT/Apache: yes. AGPL: only with source release or commercial licence
- **Where does the data go?** SaaS training = your vibration data in someone's cloud
 - fine for a PC fan; a question for pump telemetry with customer data
- **Who controls pricing/roadmap?** open-source: fork-able; SaaS: acquisition risk is real
- Ask legal *early* — retrofitting a licence change into shipped firmware is misery

Selection criteria 2 — footprint & fit

Our budget on RAK4631: **256 KB RAM / 1 MB flash** — minus BLE stack, drivers, app...

- Trees (emlearn): flash $\propto$ total node count, RAM $\propto$ #features — **O(kB)**
- Dense NN on features: weights = (inputs×units) floats — small
- CNN on raw windows: activations dominate → **RAM (not flash) is the wall**
- Interpreter frameworks add a fixed floor before the first model byte
- Rule: measure with *your* model — every framework's "typical" number is marketing

Selection criteria 3 — workflow & lock-in

- **Iteration loop length**: data → model → device → observe. EI: minutes, all-GUI. emlearn: minutes, all-scriptable. TFLM: converter friction. AlfES: manual weight plumbing
- **Reproducibility**: can you re-create today's model in 2 years?
 - scripts + pinned versions (emlearn/TFLM) vs cloud project state (EI)
- **Exit cost**: what does migrating away require? (retraining? rewriting DSP? re-labelling?)

Deep dive: emlearn

Why we go deep on this one

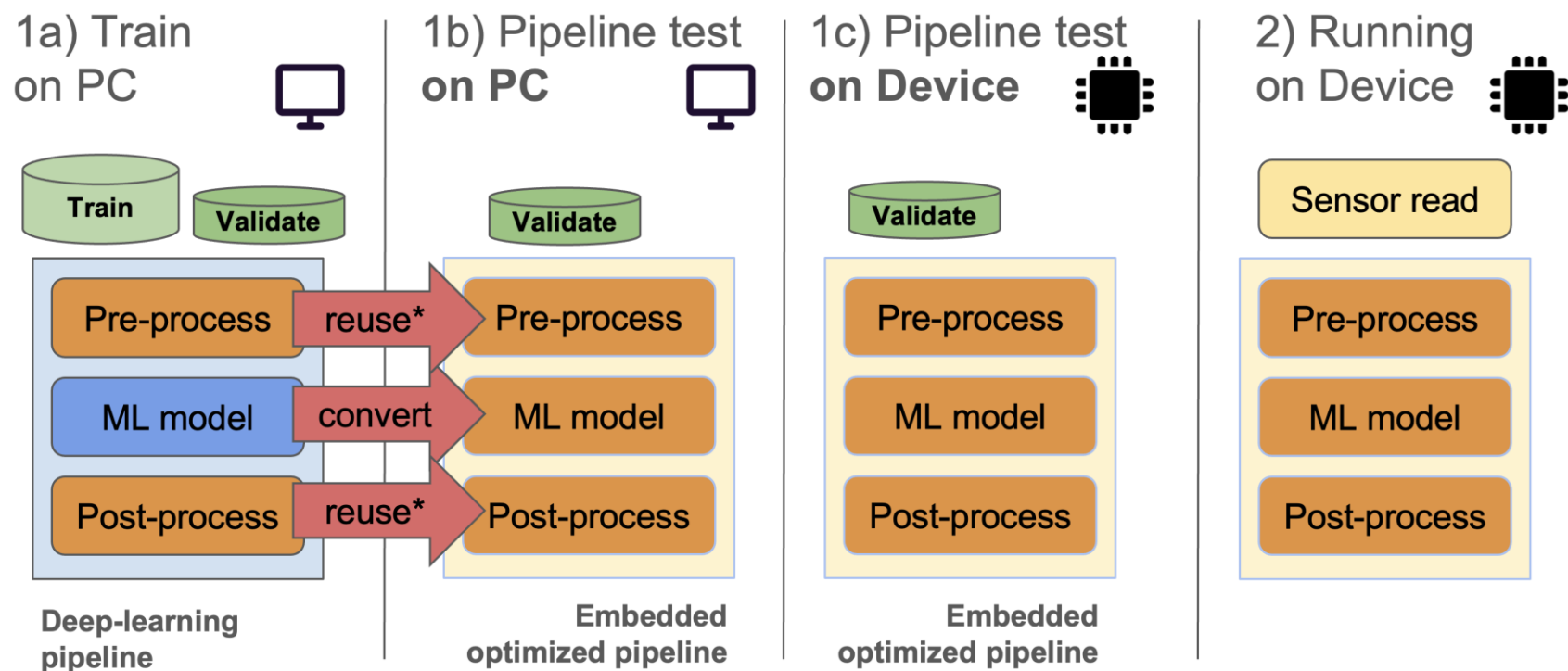
- Everything visible: input C, output C, no cloud between
- Perfect teaching vehicle for the **convert-to-C** pattern used by *all* codegen frameworks
- Directly productive: Day 3's Random Forest on your fan features deploys via emlearn
- *Slides in this section adapted from Jon Nordby's guest lecture (Aarhus, March 2026)*

emlearn: the whole process

```
0. pip install emlearn
1. train:  model = RandomForestClassifier(...).fit(X, y)
2. convert: cmodel = emlearn.convert(model, method='inline')
           cmodel.save(file='xor_model.h', name='xor_model')
3. use:    #include "xor_model.h"
           out = xor_model_predict(features, N_FEATURES);
```

- Header-only generated C99: **no dynamic allocation, portable, testable on your laptop with gcc first**
- Same code runs on host and device → validate before flashing

Deploy in steps —4-step ladder



1. Python features + Python model ✓ (notebook)
2. **C features + C model on your laptop** — same static window → same class?
3. C features + C model on the RAK4631, static test arrays
4. Live sensor data

emlearn: training tips for embedded targets

```
from sklearn.ensemble import RandomForestClassifier
# keep the model small ON PURPOSE
estimator = RandomForestClassifier(n_estimators=10, max_depth=10)
estimator.fit(X, y)
```

- **Limit model size at training time** (`n_estimators` , `max_depth` , `min_samples_leaf`)
- Consider integer inputs (int16) — enables integer-only inference paths
- Trees need **no feature scaling** — one less PC-vs-device mismatch

emlearn: cost model for tree ensembles

Resource	Scales with
RAM	$O(\text{\#features})$ — basically just the input vector
Flash	$O(\text{\#nodes}) \approx \text{trees} \times 2^{\text{depth}}$
CPU	Σ effective depth per tree (data-dependent!)

- More trees $\rightarrow$ more capacity, **less** overfitting; deeper trees $\rightarrow$ more capacity, **more** overfitting
- Tune `n_estimators` + one depth-limiting parameter with `GridSearchCV`
- `emlearn.evaluate.pareto`: find the accuracy-vs-compute **Pareto front**

emlearn in the wild

- **Soundsensing** (Nordby's company): sound/vibration condition monitoring of building HVAC — our fan lab, productised
- **Cattle health collar** (Virginia Tech): decision tree on accelerometer, LoRaWAN uplink — **< 1 mW**, 50× less power than streaming raw data
- **Breathing-rate earable** (Samsung Research): RF on audio features
- **Toothbrush timer**: M5StickC, ~500 lines of MicroPython, data collected + labelled

Exercise hand-off — Module 6

`exercises/module6-emlearn/`

1. `xor-train/` — laptop: sklearn RF on the XOR dataset → plot decision boundary → `emlearn.convert()` → `xor_model.h` (+ verify predictions in Python)
2. `xor-device/` — PlatformIO: drop the header into `include/`, classify pairs over **serial input**, LED feedback, measure latency with `micros()`
3. **El round-trip** (~15 min) — your Module 4 fan project → Studio *Deployment* → Arduino library → unzip into a PlatformIO project's `lib/` and **build** for the RAK4631 (no inference code yet — Module 7 writes that)
4. **Worksheet** — `framework-comparison.md`: fill the licence/footprint/workflow/lock-in table from the frameworks' own docs; we reconcile at the wrap-up

Bonus: circular dataset — how many trees/depth to fit a ring? · Try `method='loadable'` and diff the headers

Wrap-up: the decision in one slide

- **Prototyping / teaching / this course**: Edge Impulse for speed + **emlearn for understanding** — they compose (EI for data, emlearn for the model, your C features in between)
- **Product, classical ML, tiny budget**: emlearn (MIT, 2 kB, readable)
- **Product, NN, on-device adaptation**: AlfES (budget for the commercial licence)
- **Existing TF/Keras investment, NN-heavy**: TFLite-Micro (+ CMSIS-NN)
- In every case: **your features, your validation** (Module 5) travel with you

References & further material

- Jon Nordby — *Practical TinyML with emlearn* (Aarhus 2026)
- Fraunhofer IMS — *AlfES Workshop* decks + nRF52840 task zips (2025/2026)
- emlearn docs: emlearn.readthedocs.io · repo: github.com/emlearn/emlearn
- AlfES: github.com/Fraunhofer-IMS/AlfES_for_Arduino
- TFLite-Micro: github.com/tensorflow/tflite-micro
- Edge Impulse docs: docs.edgeimpulse.com
- Elsts & McConville (2021), *Are Microcontrollers Ready for Deep Learning-Based HAR?*